Лабораторная работа №7. Автоматизация тестирования и TDD (Test-Driven Development)

1. Цель работы

Изучить принципы автоматизации функциональных тестов, освоить паттерн Page Object Model для создания поддерживаемых тестовых сценариев, а также получить практические навыки разработки через тестирование (TDD), применяя цикл «Red-Green-Refactor» для доработки существующего приложения.

2. Теоретическая часть

2.1. Автоматизация тестирования

Автоматизация тестирования — это процесс использования программных инструментов для выполнения тестов, управления тестовыми данными и анализа ожидаемых результатов вместо ручного выполнения. Основная цель — сокращение времени регрессионного тестирования и повышение его точности.

Пирамида автоматизации тестирования — это концепция, которая описывает идеальное соотношение разных уровней тестов:

1. **Модульные тесты (Unit Tests)** — основа пирамиды. Самые быстрые, дешевые и многочисленные. Тестируют отдельные классы и методы изолированно (это было в ЛР №4).
2. **Интеграционные тесты (Integration Tests)** — проверяют взаимодействие между модулями, с базой данных, файловой системой или внешними сервисами (это было в ЛР №5).
3. **UI-тесты / Сквозные тесты (UI / End-to-End Tests)** — вершина пирамиды. Самые медленные и дорогие. Имитируют действия реального пользователя в интерфейсе приложения. Именно их мы будем автоматизировать в этой работе.

2.2. Паттерн Page Object Model (POM)

При автоматизации UI-тестов критически важно сделать тесты поддерживаемыми и устойчивыми к изменениям интерфейса. **Page Object Model** — это самый популярный паттерн для решения этой задачи .

Суть паттерна:

- Для каждой страницы или значимого компонента интерфейса (например, модальное окно, панель навигации) создается отдельный класс — **Page Object**.

- Этот класс хранит **локаторы** всех элементов на данной странице (кнопки, поля ввода, ссылки) и методы для **взаимодействия** с ними (клик, ввод текста, получение значения).
- Сами тесты используют эти классы, не зная деталей разметки (HTML, XPath, AutomationId). Если интерфейс меняется, правки вносятся только в один Page Object, а не во все тесты.

Пример структуры:

```
PageObjects/
├── LoginPage.cs      // Класс для страницы входа
│   ├── Локаторы: usernameField, passwordField, loginButton
│   ├── Методы: LoginAs(user, pass), GetErrorMessage()
├── MainPage.cs        // Класс для главной страницы после входа
│   ├── Локаторы: welcomeLabel, logoutButton
│   ├── Методы: GetWelcomeText(), Logout()
└── Tests/
    └── LoginTests.cs  // Тесты, использующие LoginPage и MainPage
```

2.3. Разработка через тестирование (TDD)

TDD — это методология разработки, при которой тесты пишутся **до** написания самого кода. Это не просто способ тестирования, а инструмент проектирования, который заставляет разработчика думать о требованиях и интерфейсе до реализации .

Цикл Red-Green-Refactor :

1. **Red (Красный)**: Напишите тест на новую функциональность. Так как функциональности еще нет, тест не будет компилироваться или упадет (будет красным). Это четко определяет цель.
2. **Green (Зеленый)**: Напишите **минимально возможное количество кода**, чтобы тест прошел. На этом этапе код может быть некрасивым или неоптимальным. Главное — получить зеленый сигнал.
3. **Refactor (Рефакторинг)**: Теперь, когда поведение защищено тестами, можно спокойно улучшать код: убирать дублирование, улучшать читаемость, оптимизировать структуру. После каждого изменения тесты перезапускаются, чтобы убедиться, что ничего не сломалось.

Материалы для самостоятельного изучения:

1. Приложение к лабораторной работе: Шпаргалка Page Object Model.
2. POM на python: https://habr.com/ru/articles/472156/.

3. FlaNium — инструмент для тестирования настольных (desktop) приложений под Windows (C#): https://habr.com/ru/companies/lanit/articles/553588/.

4. FlaUI на практике: поиск элементов, умные ожидания и борьба с динамическим UI: https://habr.com/ru/articles/943650/.

5. Python Playwright (инструмент автоматизации тестирования): https://tproger.ru/articles/sovremennyj-podhod-k-avtomatizacii-testirovaniya-veb-prilozhenij-s-pomoshhyu-python-playwright-242777.

6. Автоматизация UI тестов на Cypress (JS): https://habr.com/ru/articles/909568/.

7. Кратко про TDD: https://tquality.ru/blog/razrabotka-cherez-testirovanie-tdd/.

8. Пример того, как TDD правильно использовать: https://habr.com/ru/companies/sberbank/articles/851052/.

3. Задание на лабораторную работу

Работа состоит из двух частей. В качестве объекта тестирования используется приложение, разработанное (или доработанное) в предыдущих лабораторных работах (система авторизации пользователей).

Задание 1. Автоматизация UI-сценариев с Page Object Model

1. Выбор инструментария:
   - В зависимости от типа вашего приложения (консольное, WinForms, WPF, веб-интерфейс), выберите инструмент для UI-автоматизации.
     - Для **WinForms/WPF** рекомендуется использовать **FlaUI, FlaNiUi** (библиотека для C#) или WinAppDriver.
     - Для **веб-приложений** рекомендуется использовать **Selenium WebDriver, Playwright, Cypress**.
     - Также могут использоваться альтернативные инструменты, который студент считаем подходящим для своего языка программирования и стэка технологий.
2. Анализ и создание Page Objects:
   - Проанализируйте функционал вашего приложения с точки зрения пользователя.
   - Выделите ключевые страницы/экраны: форма входа, форма регистрации, главное окно после входа, панель администратора для смены ролей.
   - Для каждой страницы создайте отдельный класс. В конструкторе класса принимайте драйвер автоматизации.

- Используя инспектор элементов (например, FlaUInspect или встроенные средства браузера), определите уникальные идентификаторы элементов управления (AutomationId, Name, CSS-селекторы) и сохраните их как поля класса.
- Реализуйте методы, инкапсулирующие взаимодействие с элементами (например, LoginPage.FillLoginForm("user", "pass")).

3. Написание автоматизированных тестов:
   - Разработайте автоматизированные сценарии, которые покрывают позитивные и негативные пути из вашего чек-листа (ЛР №6). **Минимум 3 сценария**.
     - *Сценарий 1 (Позитивный):* Успешная авторизация существующего пользователя.
     - *Сценарий 2 (Негативный):* Попытка входа с неверным паролем -> проверка появления сообщения об ошибке.
     - *Сценарий 3:* Регистрация нового пользователя и проверка, что он может войти в систему.
   - В тестах используйте созданные Page Object'ы. Тесты должны быть читаемыми и не содержать прямых обращений к UI-элементам.

Задание 2. Разработка новой функции через TDD (Red-Green-Refactor)

Допустим, в вашем приложении появилось новое требование: "Пароль пользователя должен считаться надежным, если его длина не менее 8 символов и он содержит хотя бы одну цифру". Ваша задача — реализовать метод bool IsPasswordStrong(string password), следуя TDD.

1. Фаза «Red»:
   - В проект с модульными тестами (из ЛР №4) добавьте новый тестовый класс PasswordPolicyTests.
   - Напишите **первый падающий тест**. Начните с простого случая: проверьте, что пароль короче 8 символов возвращает false.
   - Запустите тесты. Новый тест должен упасть (Red), так как метод еще не реализован.

csharp

```
// Файл: PasswordPolicyTests.cs
[Test]
public void IsPasswordStrong_PasswordShorterThan8_ReturnsFalse()
{
   // Arrange
```

```
    var service = new AuthenticationService(); // Или новый класс
    string weakPassword = "short";

    // Act
    bool result = service.IsPasswordStrong(weakPassword);

    // Assert
    Assert.IsFalse(result);
}
```

2. Фаза «Green»:
   - Напишите минимальную реализацию метода, чтобы этот тест прошел. Пока можно просто вернуть false всегда.

```csharp
// Файл: AuthenticationService.cs
public bool IsPasswordStrong(string password)
{
    return false; // Минимально для прохождения теста
}
```

3. Новый цикл (Red для позитивного случая):
   - Напишите следующий тест — для пароля длиной 8 символов, но без цифры. Ожидаете false.
   - Запустите — тест упадет, потому что метод всегда возвращает false. Это хорошо.
   - Модифицируйте метод, чтобы проверять длину, но по-прежнему возвращать false, если нет цифры.
   - Запустите тесты — теперь первый тест (на длину) и второй (на отсутствие цифры) должны стать зелеными.
4. Фаза «Refactor»:
   - Теперь, когда у вас есть защита тестами, улучшите код метода. Объедините проверки, используйте LINQ (Any(char.IsDigit)), добавьте проверку на null. Убедитесь, что тесты по-прежнему проходят .
5. Финализация:
   - Добавьте тест на успешный случай (пароль «StrongPass1») и, если нужно, доработайте метод. Отразите в отчете все шаги с кодом и результатами запуска тестов.

4. Содержание отчёта

Отчёт должен содержать:

Отчёт должен содержать:

1. Цель работы.
2. Описание автоматизации (Задание 1):
   - Обоснование выбора инструмента автоматизации (FlaUI, Selenium и т.д.).
   - Диаграмма или схема, описывающая структуру Page Object'ов.
   - Листинги ключевых Page Object классов (с комментариями).
   - Листинги автоматизированных тестов (не менее 3-х).
   - Скриншоты успешного прохождения автотестов.
3. Описание разработки через тестирование (Задание 2):
   - Пошаговое описание цикла Red-Green-Refactor.
   - Скриншот 1: Первый написанный тест и ошибка компиляции/падения (Red).
   - Скриншот 2: Минимальный код, сделавший тест зеленым (Green).
   - Скриншот 3: Финальный код метода после рефакторинга и прохождения всех тестов.
   - Листинги всех тестов для функции IsPasswordStrong.
4. Выводы: Сравните ручное и автоматизированное тестирование. Опишите свои впечатления от TDD: помог ли этот подход лучше понять требования и спроектировать интерфейс?

5. Контрольные вопросы

1. В чем преимущества и недостатки автоматизированного тестирования перед ручным?

2. Что такое пирамида автоматизации тестирования и почему UI-тестов должно быть мало?

3. Объясните паттерн Page Object Model. Какие проблемы он решает?

4. Какие инструменты для UI-автоматизации Windows-приложений вы знаете?

5. Что такое «Flaky» тест и как с ним бороться?

6. Опишите цикл Red-Green-Refactor. Что происходит на каждом этапе?

7. Чем TDD отличается от традиционного подхода «Сначала код, потом тесты»?

8. Какие преимущества дает TDD для дизайна кода?

9. Что такое "минимальная реализация" в контексте TDD?

10. Как TDD помогает при рефакторинге?

11. Что такое регрессионное тестирование и как автоматизация помогает в нем?

12. Какие метрики можно использовать для оценки качества автоматизированных тестов?
13. В чем разница между сквозным (E2E) тестированием и модульным?
14. Нужно ли автоматизировать все ручные тесты? Почему?
15. Как организовать структуру тестового проекта для UI-автоматизации?